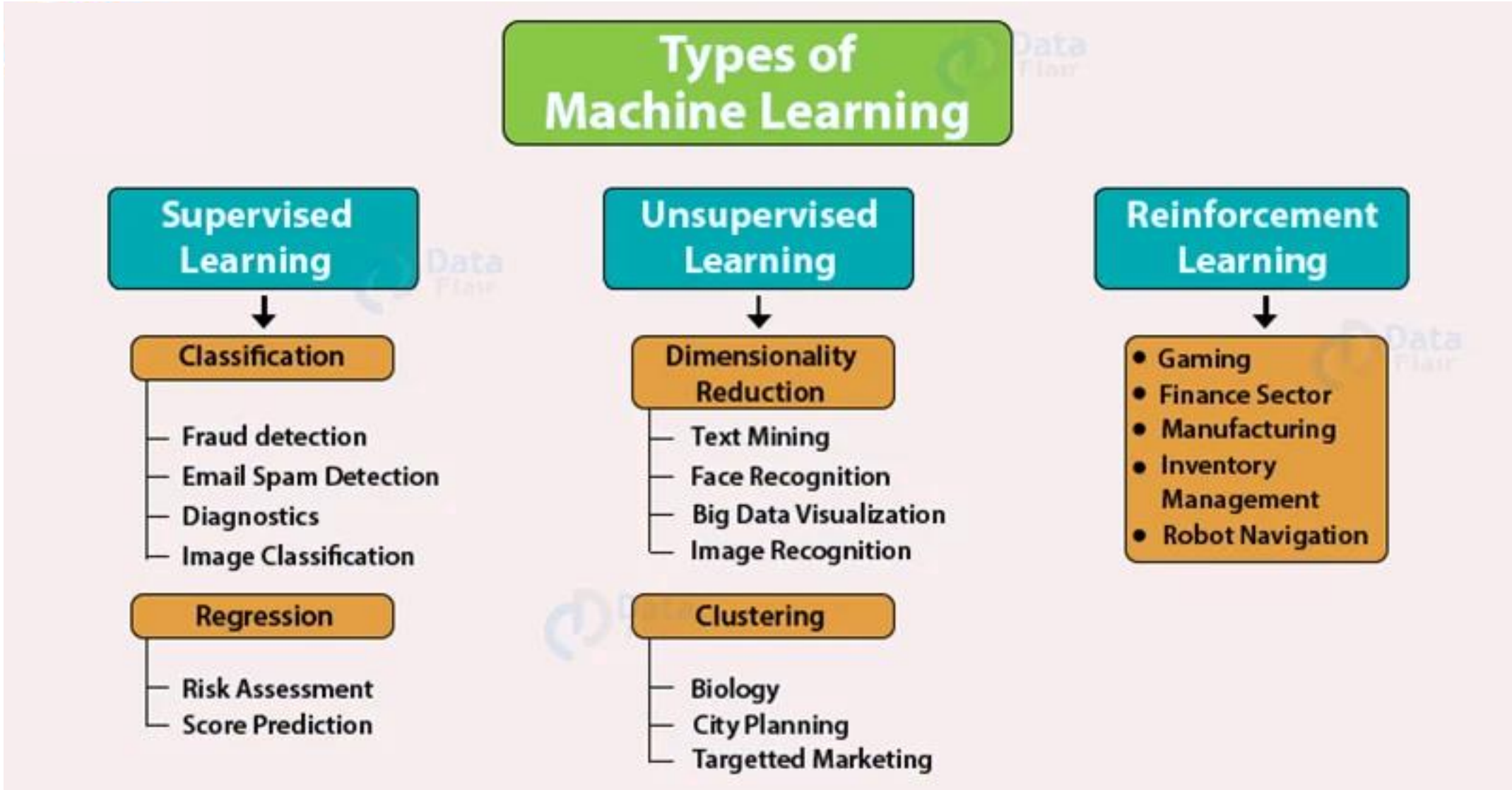
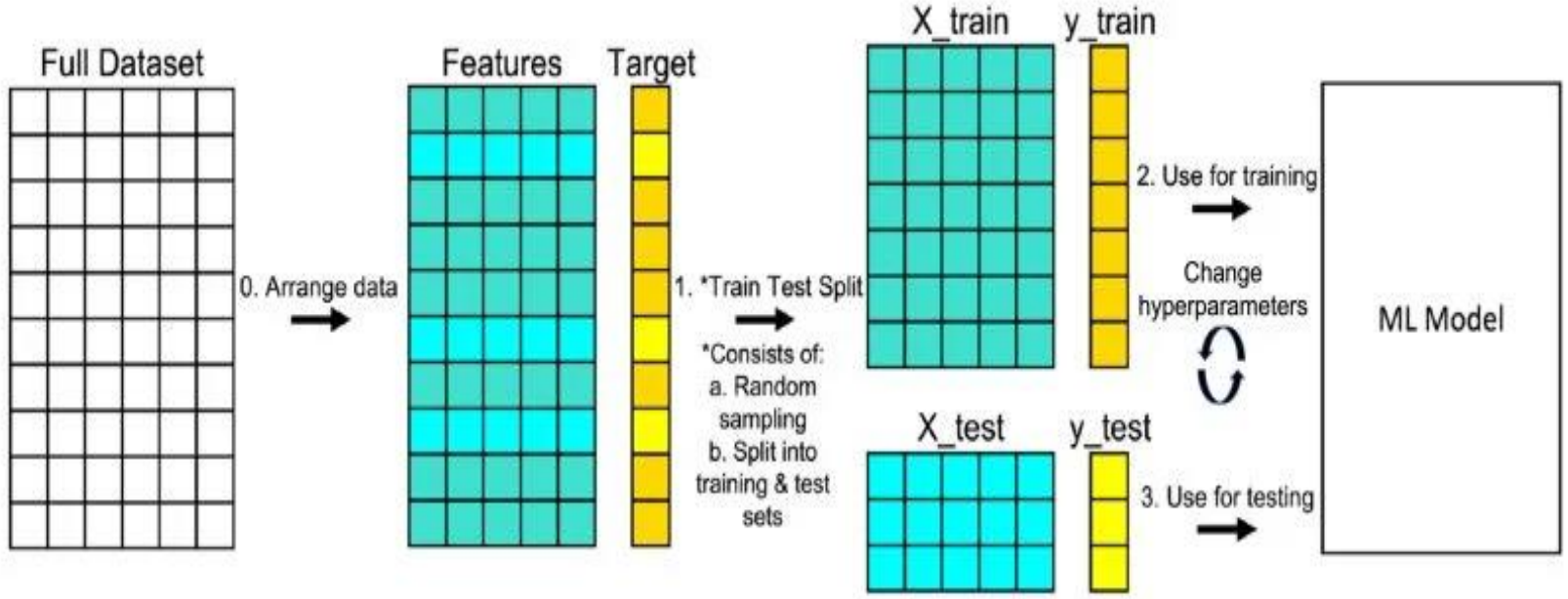
Introduction to Machine Learning


Types of Machine Learning
Supervised Learning
Classification
Fraud detection
Email Spam Detection
Diagnostics
Image Classification
Regression
Risk Assessment
Score Prediction
Unsupervised Learning
Dimensionality Reduction
Text Mining
Face Recognition
Big Data Visualization
Image Recognition
Clustering
Biology
City Planning
Targetted Marketing
Reinforcement Learning
Gaming
Finance Sector
Manufacturing
Inventory Management
Robot Navigation

What is a Classification Problem

Classification is a type of **supervised machine learning**.:

- A **target variable** you want to predict
- A **set of historical data** where that target **label** is **known**
- **New data** where that target variable is **unknown**
- A **model** is a mathematical object that takes data *where the label is **unknown** and assigns it a label*



What is a Model?

A model is a mathematical object that

- **takes** your training data,
- learns **a function mapping data** -> labels,
- and then is **able to take** in unlabeled data and **assign them** a label

f(some_new_data) => new_label

f(x) = x_wealth > 100,000

f(x) is a model!

Here, f (some_new_data) represents a function that takes new data as input.

What is Machine Learning

Raw data is not actionable without effective analysis.

Machine learning, a subset of artificial intelligence (AI), enables systems to learn from data and uncover valuable patterns.

- These insights support predictions and enhance complex decision-making.

Different Types of Machine Learning

1. Supervised Learning
2. Unsupervised Learning
3. Semi-supervised Learning
4. Reinforcement learning.

Each form of Machine Learning has differing approaches, but they all follow the same underlying process and theory.

Terminology



- **Dataset:** A collection of data points containing features relevant to solving a problem.
- **Features:** Key data points/attributes fed into a machine learning algorithm for learning.
- **Algorithm:** A set of rules that the machine follows to learn from data and make predictions or decisions.
  - It's the process or method used for training a model.
- **Model:** a function that takes features (input data) and maps them to predicted targets (output).
  - Learns the relationship between the input data and the desired output during training, and
    - uses that relationship to make predictions on new, unseen data.
    - For instance, a decision tree algorithm produces a decision tree model.

ML Process

Data Collection: Gather the data that the algorithm will learn from.

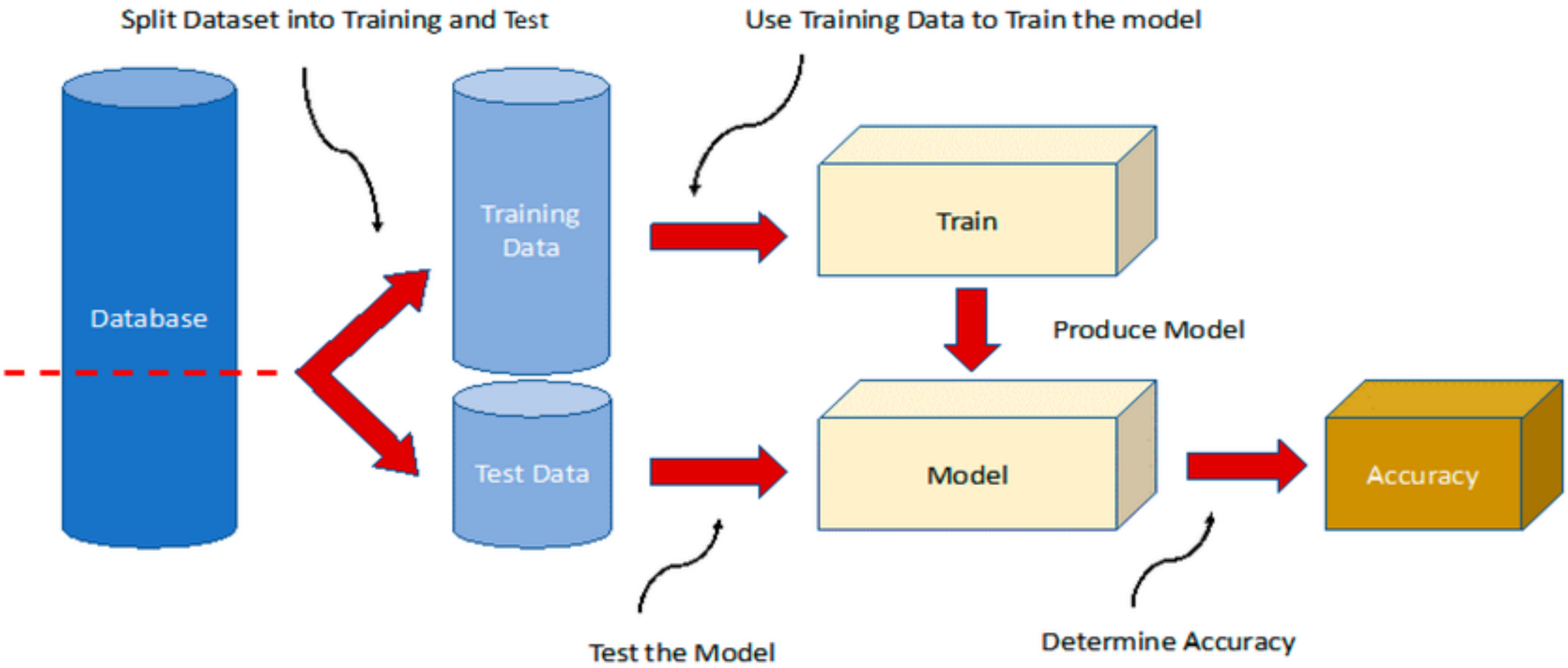
Data Preparation: Format and process the data to optimize it, including feature extraction and dimensionality reduction.

Training: Also known as the fitting stage, where the machine learning algorithm learns from the collected and prepared data.

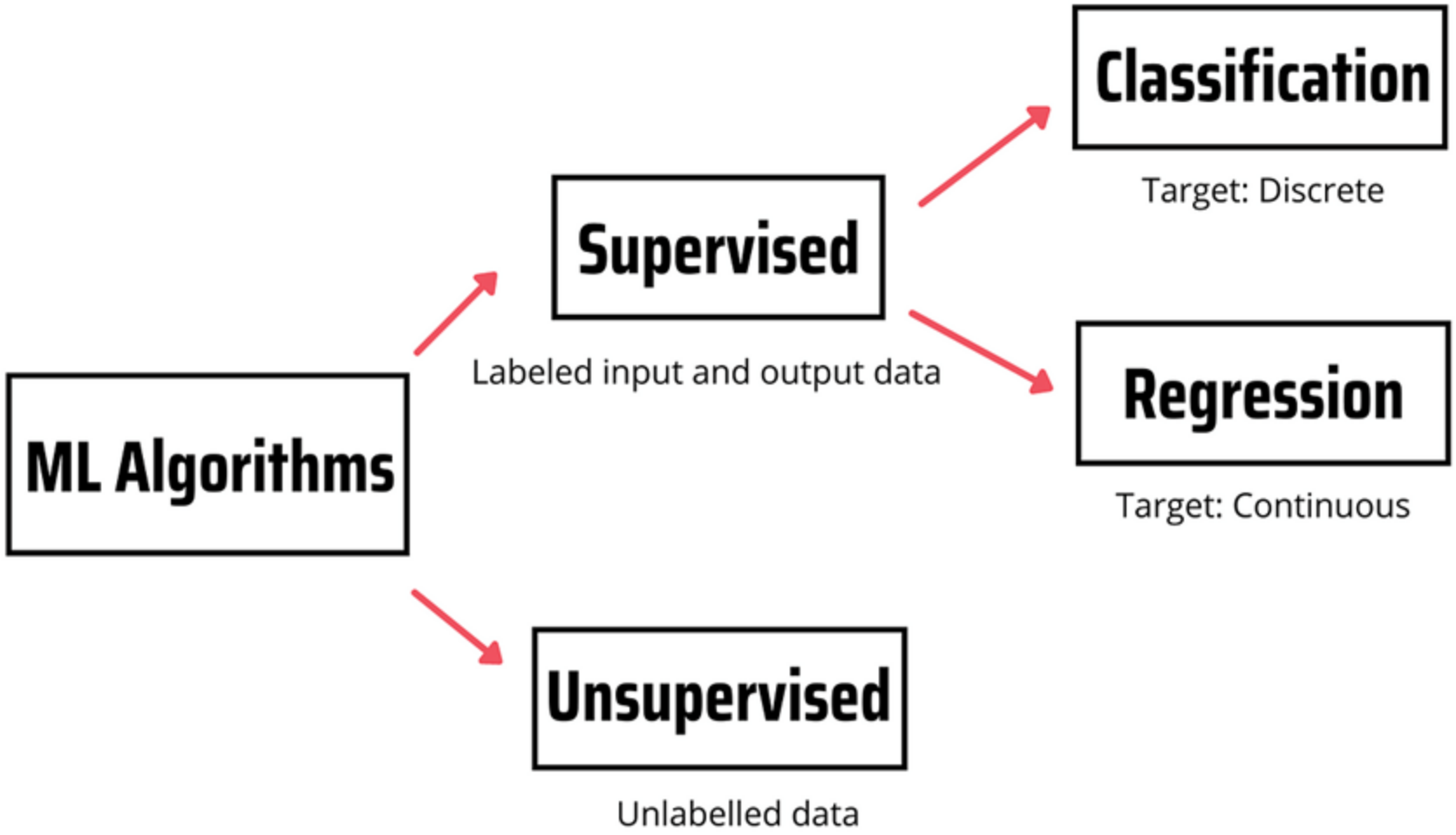
Evaluation: Test/Assess the model on test data to see how well it performs.

Tuning: Fine tune/ Adjust the model to maximise its performance.

ML Process


Split Dataset into Training and Test
Use Training Data to Train the model
Database
Training Data
Test Data
Train
Produce Model
Model
Accuracy
Test the Model
Determine Accuracy

Types of Learning

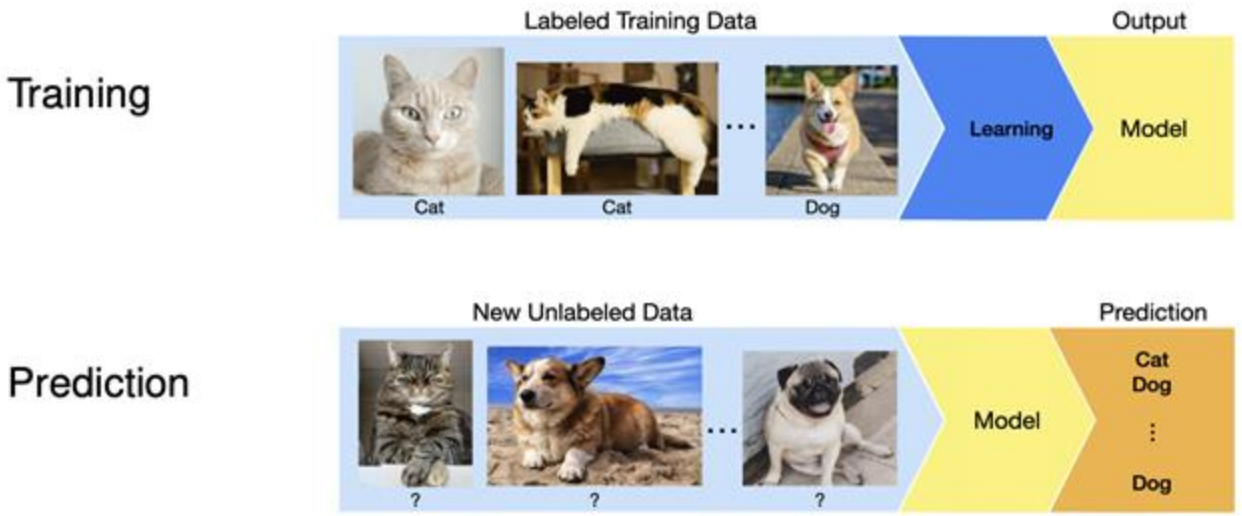


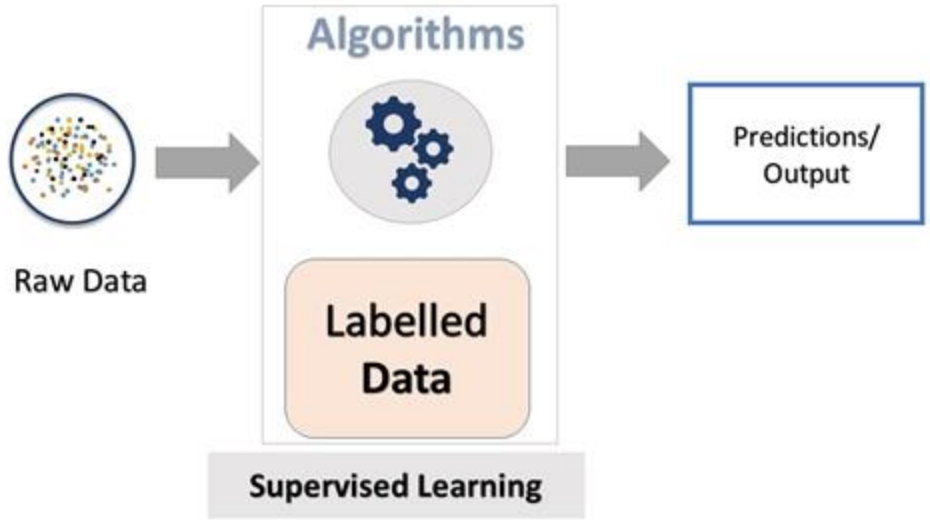
Supervised Learning

Supervised Learning: The algorithm learns from labeled data, where each example in the training set is associated with a known target or output or label.

Goal: Learn the functional relationship (mapping/rules) between inputs (features) and output (target).
Example: Classify emails as spam or not using a labeled dataset.

- Used for tasks like classification and regression.

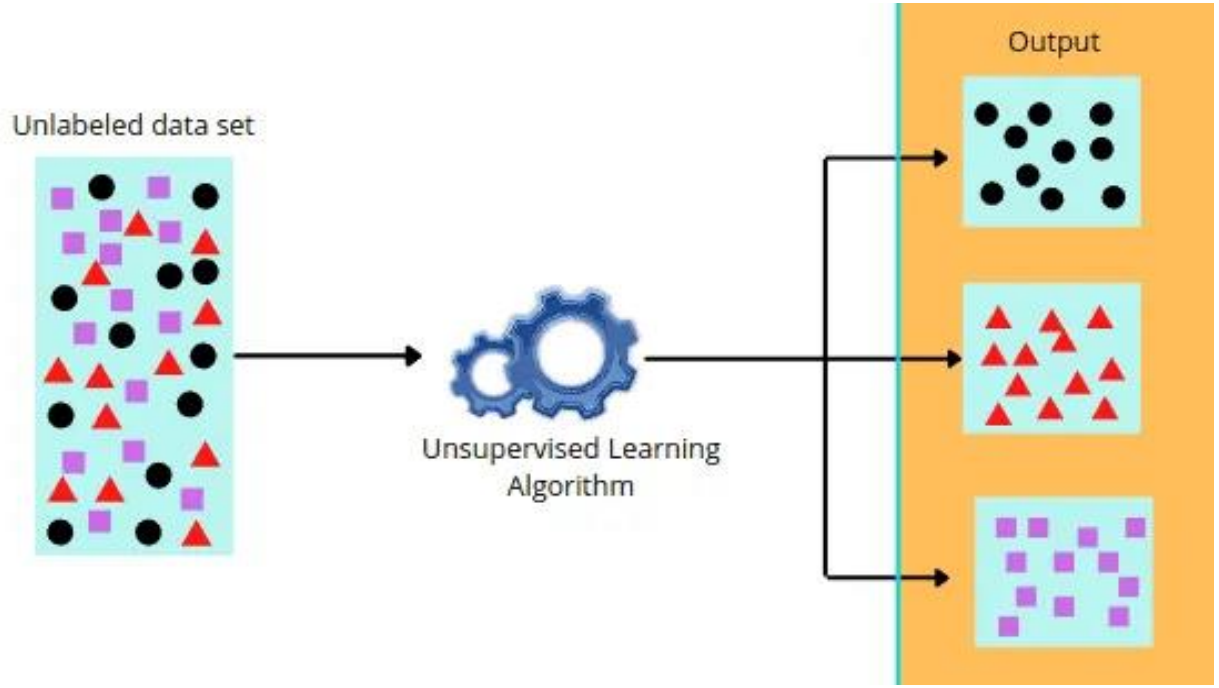




Unsupervised Learning

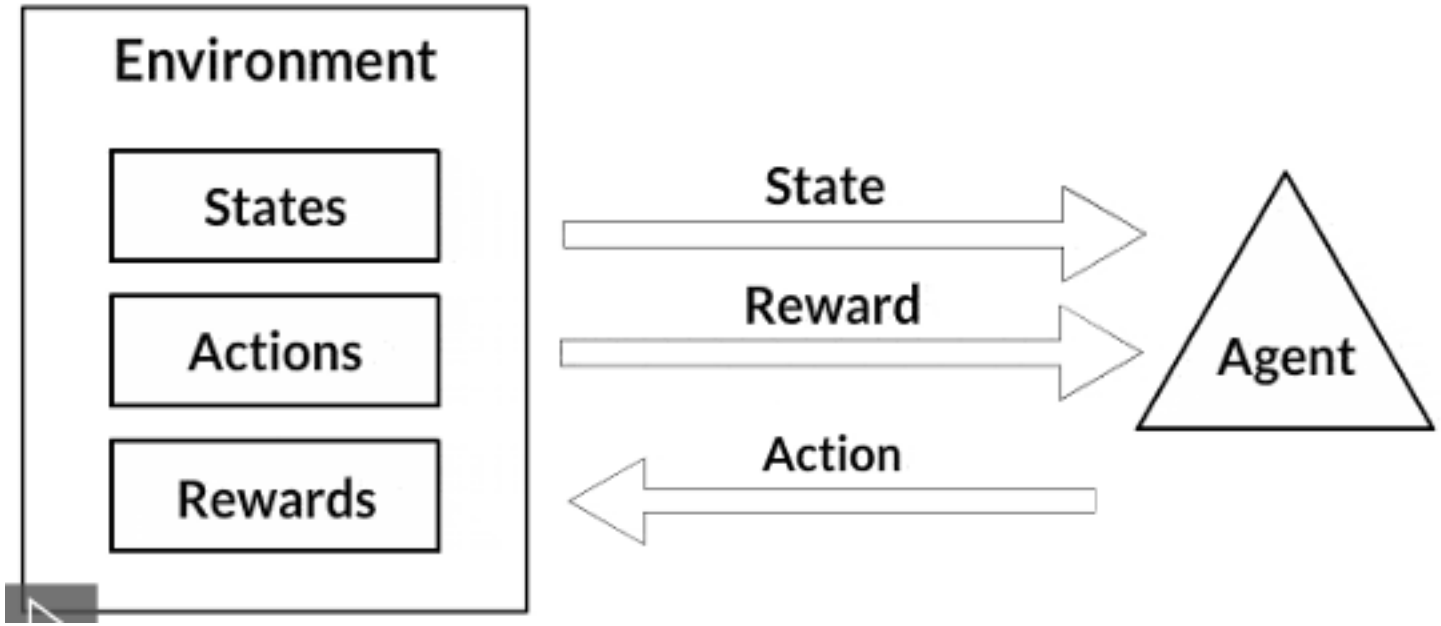
Unsupervised Learning: **The algorithm learns from unlabeled data and tries to identify patterns or structures within the data**, such as clustering similar data points or reducing data dimensionality.

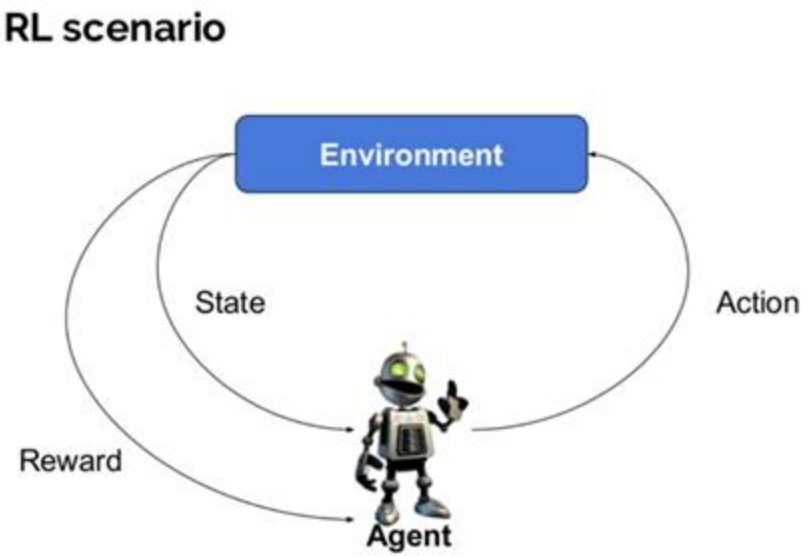
- **Example:** Group similar customer profiles together in a retail dataset without knowing in advance what those groups represent.



Reinforcement Learning

- **Reinforcement Learning:** involves training an agent to interact with an environment and learn from the feedback it receives.
  - The agent takes actions, receives rewards or penalties based on its actions, and adjusts its behavior over time to maximize the cumulative reward. Commonly used in applications such as game playing, robotics, and autonomous systems.





KNN(K Nearest Neighbour)

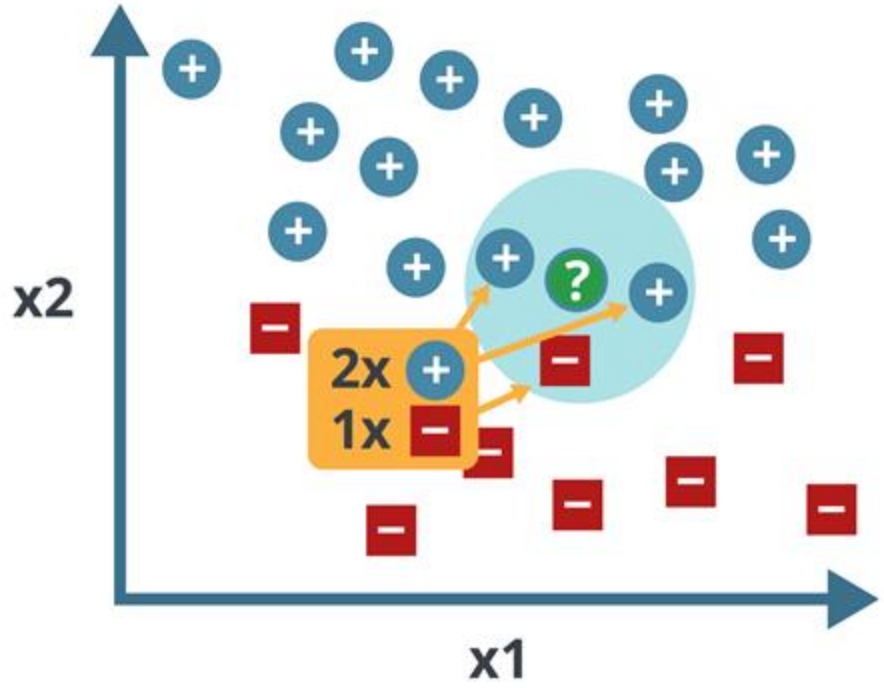
A simple algorithm in that can make predictions about an new example based on the labels of other examples "near" it.

KNN - K Nearest Neighbour

A **Instance-based learning models** simply store training examples in memory and utilize those examples on-demand to make predictions for a new, previously unseen example.

- K is also known as a **hyperparameter**: represents the number of neighboring examples that will be used to make a prediction.
  - This number is usually determined based on domain knowledge, heuristics, or experimentation, though experimentation is recommended.

Too low of a K or too high of a K can both lead to poor performance.

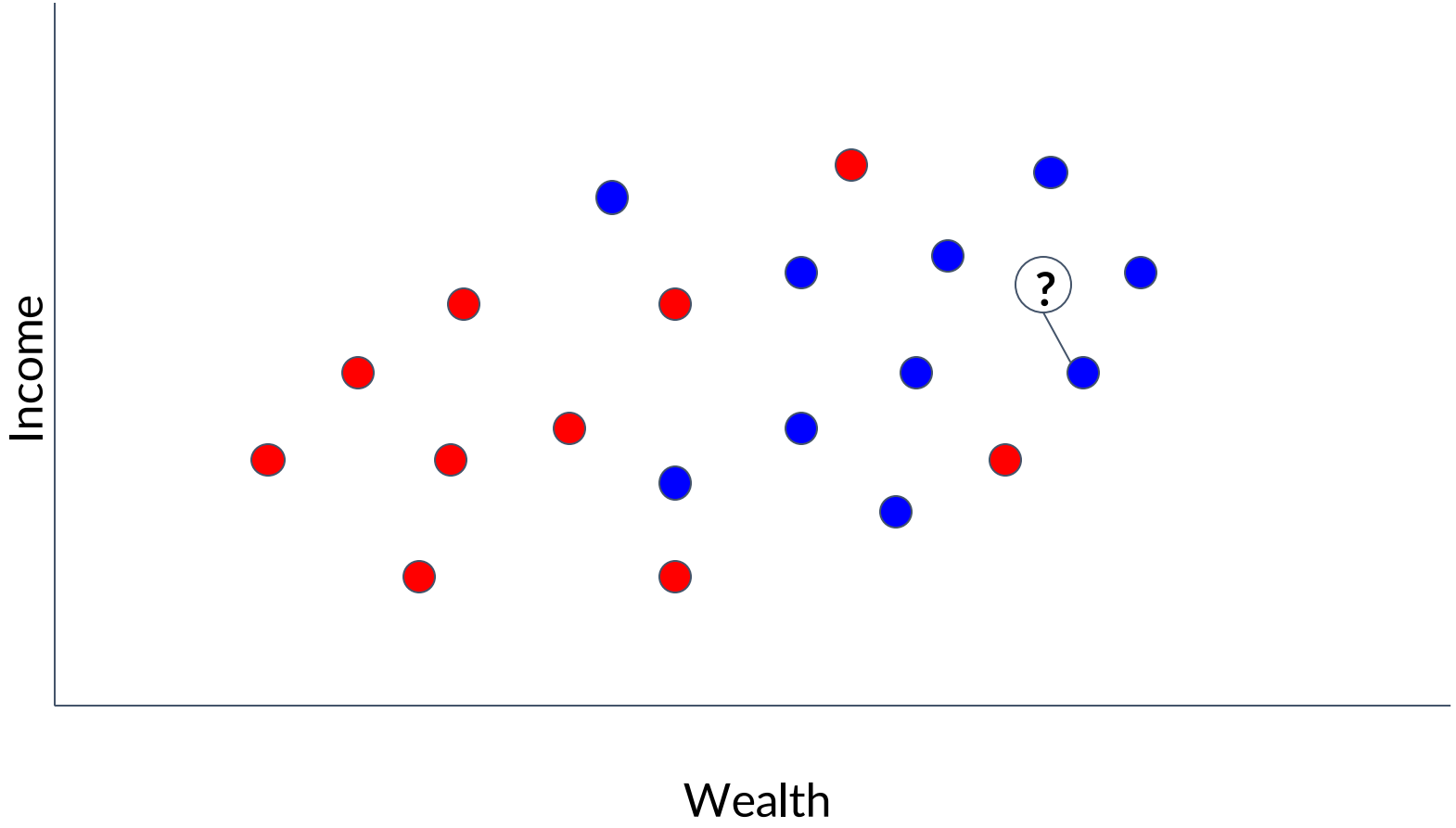


example: Let's Dumb It Down Even Further

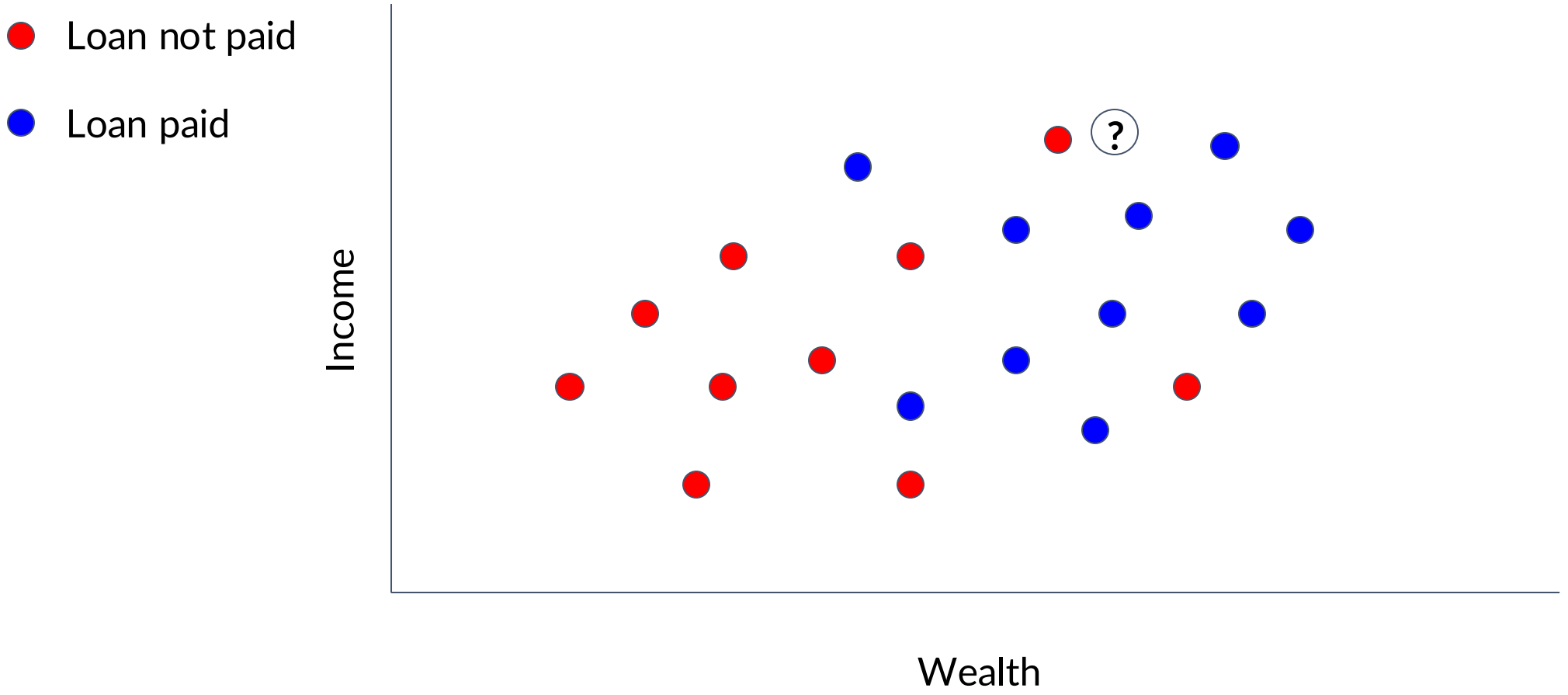
Plot the datapoint:

- Loan not paid
- Loan paid

Predict the nearest neighbor for the new datapoint and take the vote if it is a classification problem



Let's Dumb It Down Even Further



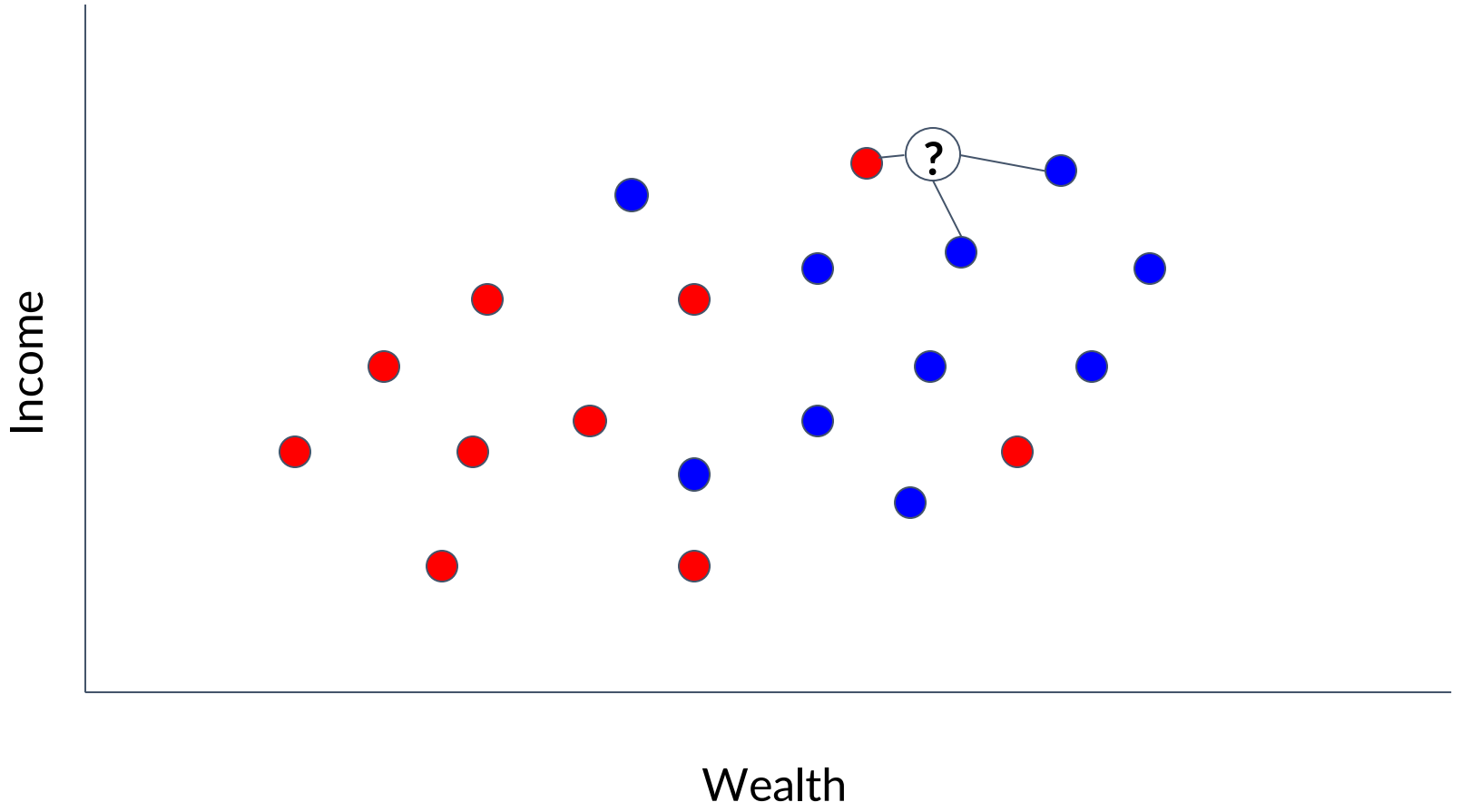
How to improve accuracy? Let's look at more than one neighbours

Let's Dumb It Down Even Further

- Loan not paid
- Loan paid

For classification: the new point is assigned to the most common class among the k neighbors (Majority Voting).

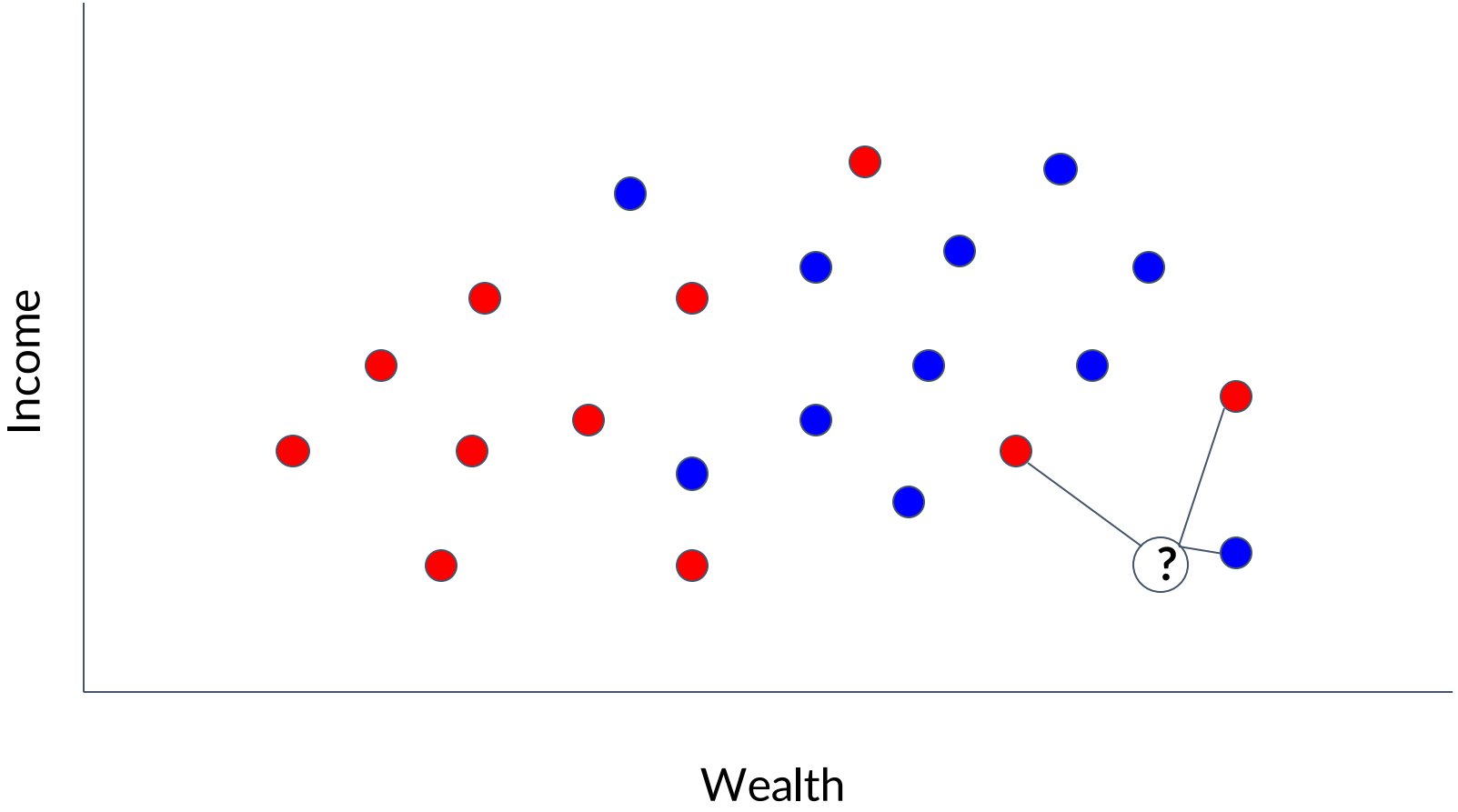
For Regression: Average the 3 nearest neighbors.



Let's Dumb It Down Even Further

- Loan not paid
- Loan paid

What about this? Do you want to classify as “blue” or “red”?



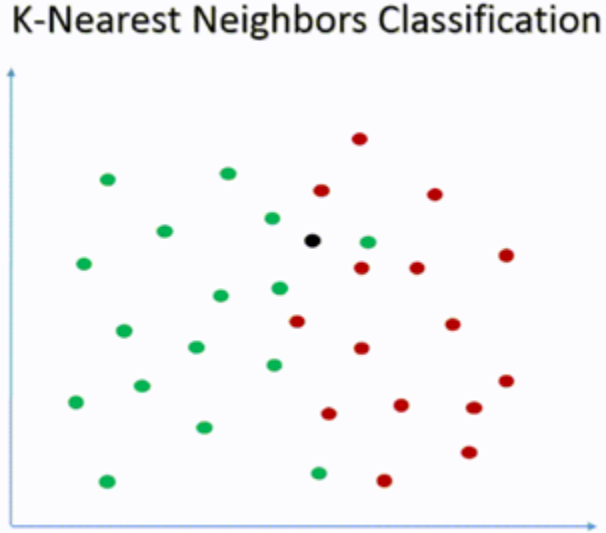
The k-nearest neighbors (KNN)

We just talk about “the k-nearest neighbors (KNN)” algorithm (Standard KNN)

- A simple, majority-based approach.
- An easy-to-implement supervised ML algorithm for solving classification and regression problems.

Simple idea

- **Store** all training examples in memory.
- **For a new example**, find its nearest neighbors.
- **Predict** based on the most common label among neighbors.



Components of a k-NN Classifier

- **Distance metric**
  - How do we measure distance between data points/ instances?
- **The k hyperparameter**
  - How large a neighborhood should we consider?
  - **The "K" in KNN:** a user-defined parameter that specifies the number of neighboring data points to consider when making predictions.
    - Choosing "K": K is typically an odd number to prevent ties in decision-making.

Hyperparameter

A configurable setting you choose for a model before it learns.

- **Set before training** (e.g., K in KNN).
- **Different from parameters**, which are automatically learned during training.
- **Example:** K in KNN
  - Determines how many nearest neighbors to consider when making predictions.

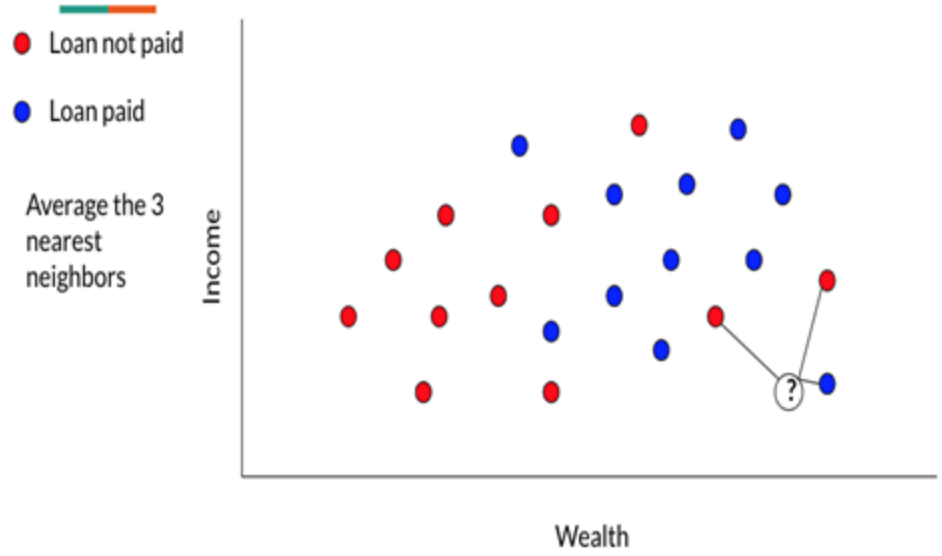
In general, in KNN

Given the set K of the K nearest neighbors and point p

- KNN Algorithm: KNN identifies the K most similar individuals (neighbors) to the person for whom we want to make a prediction (point p).

Let's represent loan repayment behavior or result using labels:

a. **Label -1:** Represents someone who did not pay back the loan.
b. **Label 1:** Represents someone who did pay back the loan.



KNN Algorithm

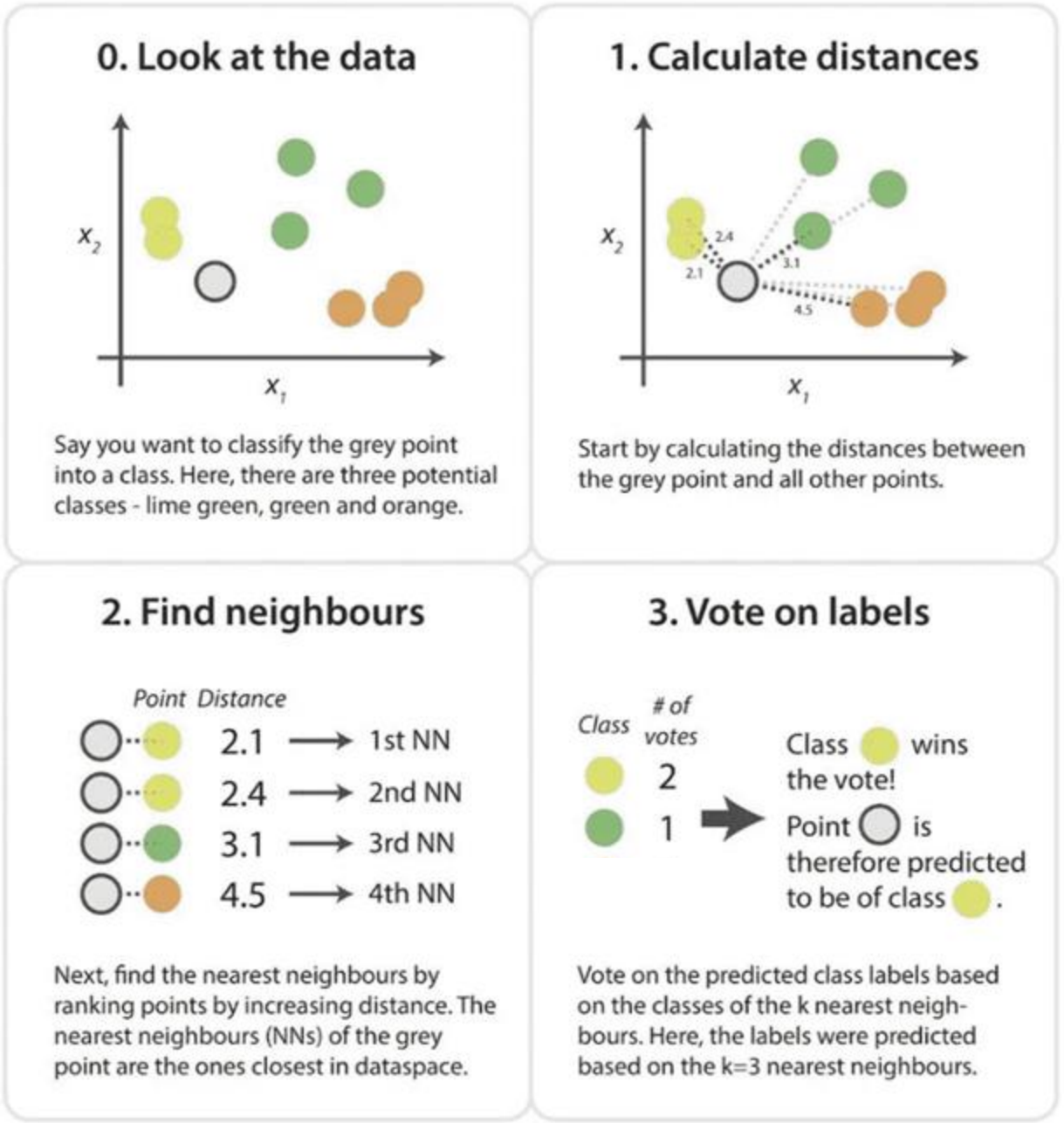
Choose K: Select the number of neighbors (K).

Calculate Distance: Compute the Euclidean distance between the new point and all training points.

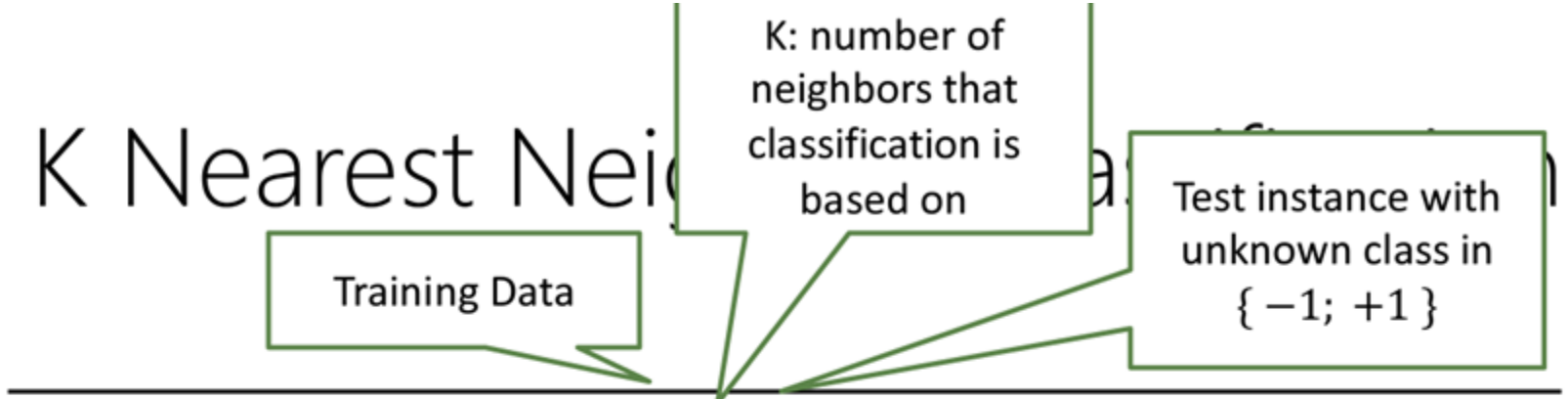
Find Neighbors: Identify the K nearest neighbors based on the calculated distances.

Count Categories: Count how many neighbors belong to each category.

Assign Category: Assign the new point to the category with the most neighbors.



KNN Algorithm Pseudocode



Algorithm 3 KNN-PREDICT($\mathbf{D}$, K, $\hat{x}$)

```
S ← [ ]
for n = 1 to N do
    S ← S ⊕ ⟨d(x_n, x̂), n⟩          // store distance to training example n
end for
S ← SORT(S)                          // put lowest-distance objects first
ŷ ← 0
for k = 1 to K do
    ⟨dist, n⟩ ← S_k                  // n this is the kth closest data point
    ŷ ← ŷ + y_n                      // vote according to the label for the nth training point
end for
return SIGN(ŷ)                       // return +1 if ŷ > 0 and −1 if ŷ < 0
```

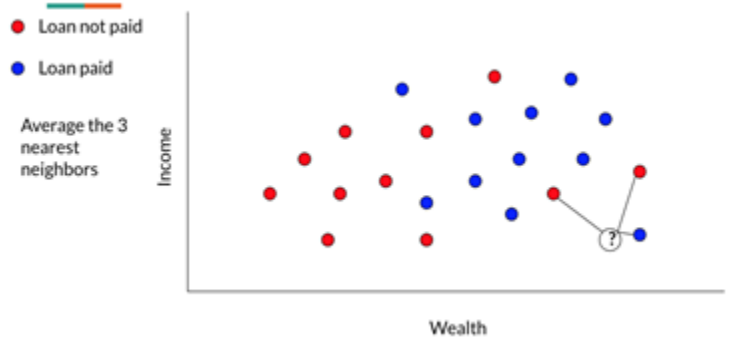
Variations on k-NN: Weighted voting

Standard KNN: All neighbors have equal weight—treats all neighbors the same, no matter their distance.

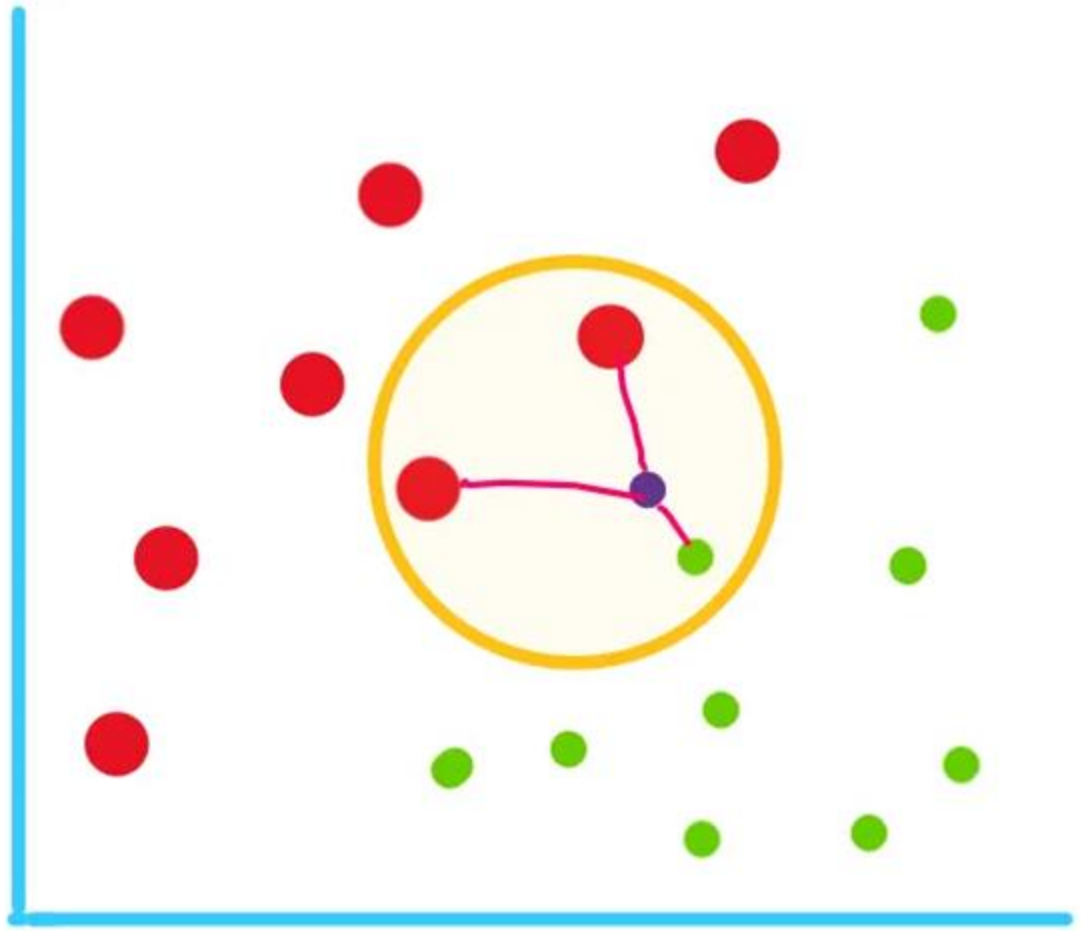
- Ex: If K=5 and the nearest neighbors are $\{1,1,1,-1,-1\}$, the prediction would be 1 (majority class).

Extension: **Weighted KNN**: Neighbors are weighted by the inverse of their distance—closer neighbors have more influence (Next Topic).

Weighted KNN is Similar to KNN, but the nearest k points are assigned a weight based on their distance.



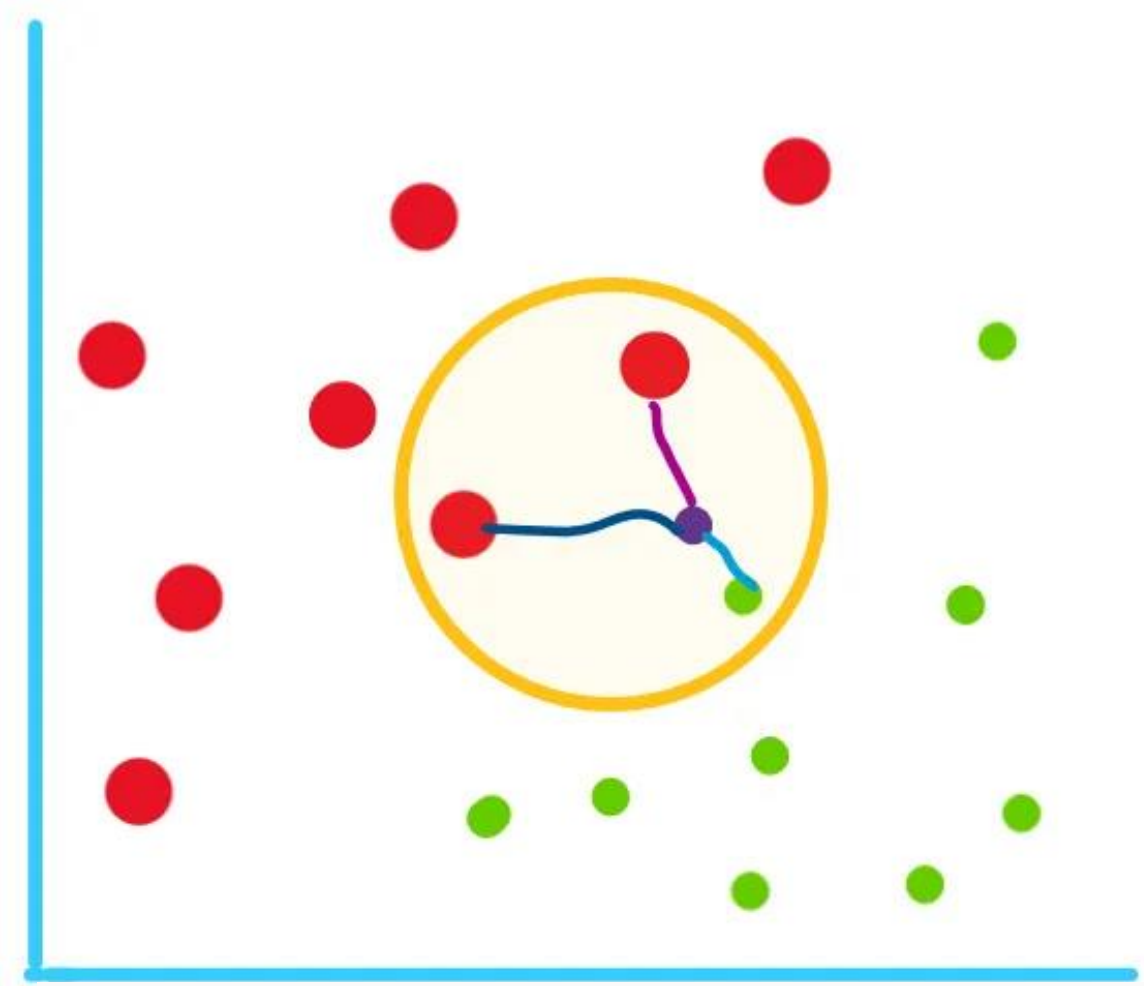
Why Weighted KNN instead of a standard KNN ?



When using basic k-NN algorithm, the new point is predicted to be ‘red’ class whereas in real, the point is closer to the ‘green’ class

This basic k-NN algorithm works quite well in many cases, but it **treats all neighbors equally**.

Why Weighted KNN instead of a standard KNN ?



In weighted k-NN, as we consider the weights of the points the predicted value will be 'green' as the point has higher weight compared to the other points.

Weighted k-NN introduces the concept of **assigning different weights to neighbors based on their proximity** to the query point, which can lead to improved performance.

The intuition behind weighted KNN is to give more weight to the points which are nearby and less weight to the points which are farther away.

Summary: KNN vs Weighted KNN

KNN	**Weighted KNN**
It treats all neighbors equally	Instead of treating all k-nearest neighbors equally, assign different weights to them based on their distance from the query point.
For classification tasks: count the occurrences of each class among the k-nearest neighbors and choose the class with the **highest count(mode)** as the predicted class.	**For classification tasks:** when determining the predicted class, the contributions of neighbors are scaled by their weights. Closer neighbors have a stronger influence on the prediction.
For regression tasks: take the **average (mean)** of the values associated with the k-nearest neighbors as the predicted value.	**For regression tasks:** the values associated with neighbors are multiplied by their weights before averaging, giving more importance to closer neighbors in the prediction.

Weighted KNN Equation

Weighted KNN not only considers the nearest neighbors but also how close or far away they and assigns different weights to the K nearest neighbors based on their distances to point p.

- How Assign? the weights are inversely proportional to the distances
- Closer neighbors have a stronger influence on the prediction.
  - Why? closer neighbors get higher weights, while farther neighbors get lower weights

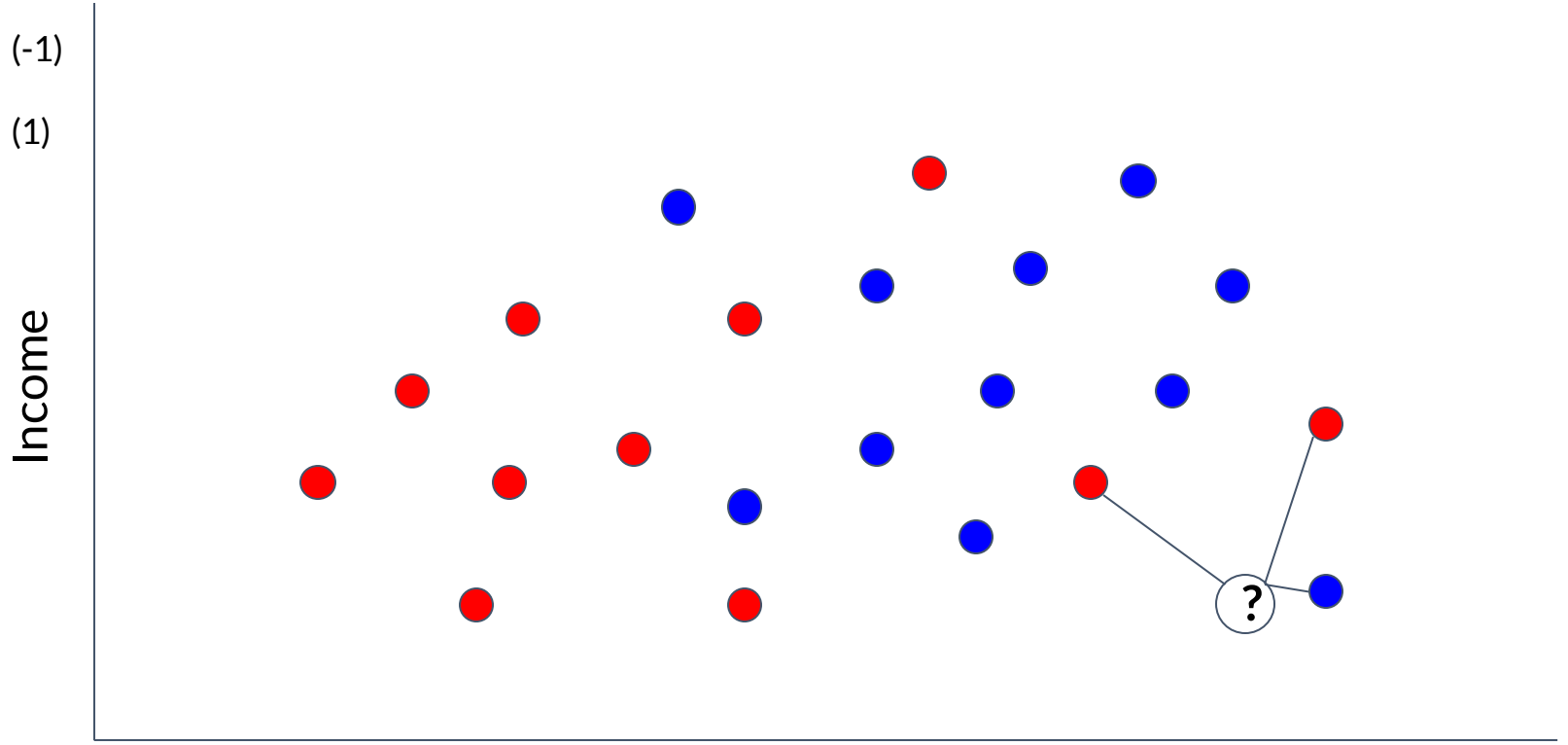
label (or value) of the k-th nearest neighbor.

$$y = \sum_{k \in K} \frac{1}{dist(k,p)} * k_{label}$$

predicted label or value for a new data point p.

Let's Dumb It Down Even Further

- Loan not paid (-1)
- Loan paid (1)

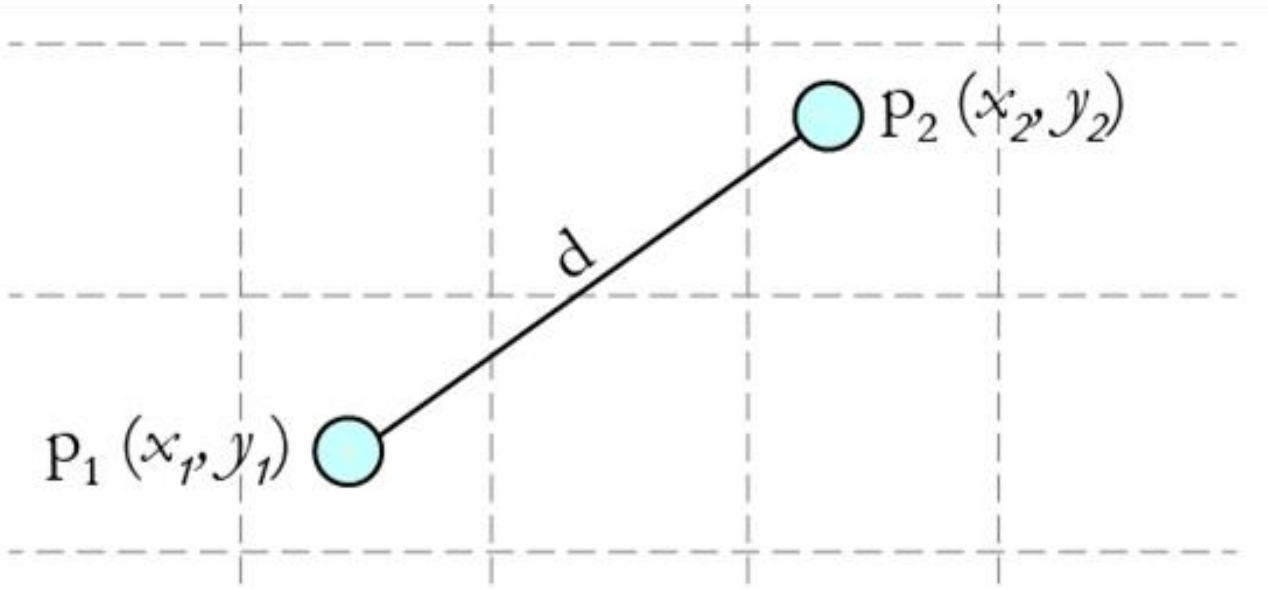


$$y = (\frac{1}{bigDistance} * -1) + (\frac{1}{bigDistance} * -1) + (\frac{1}{smallDistance} * 1)$$

Hold on--What do we even mean by distance?

Distance is a crucial concept in both KNN and Weighted KNN.

- It measures the similarity or proximity between data points.
  - the numerical measure of how far apart two data points are in the feature space.



Why Distance Matters?

Many of the Supervised and Unsupervised machine learning models such as K-Nearest Neighbor and K-Means depend upon the distance between two data points to predict the output.

Example:

In Weighted KNN, the distance between a data point (e.g., point p) and its K nearest neighbors determines the weighting.

- **Closer** neighbors have higher influence, while **distant neighbor**s have less impact.

Different kind of distance functions

The choice of distance metric depends on the nature of the data and the problem at hand.

Different kind of distance functions:

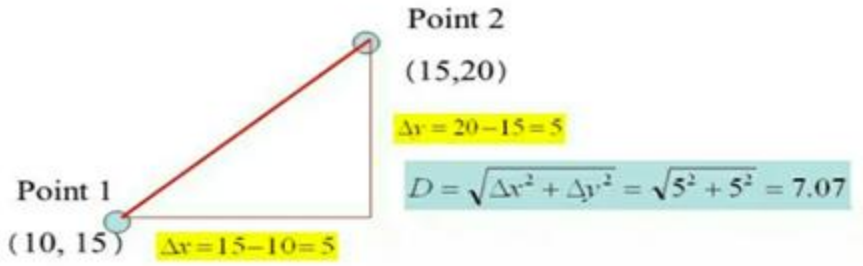
- Manhattan Distance
- Euclidean Distance
- Hamming distance
- Cosine Similarity

Euclidean Distance (L2 Distance) Most common one

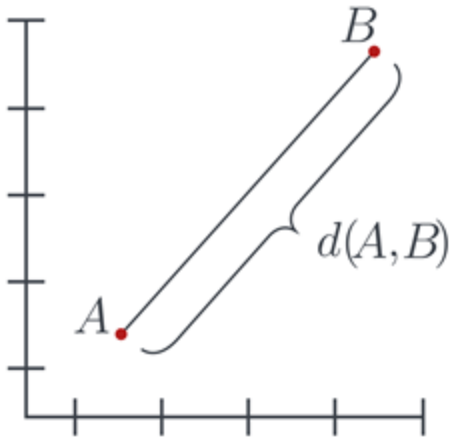
Represents the shortest (straight-line or geometric) distance between two points.

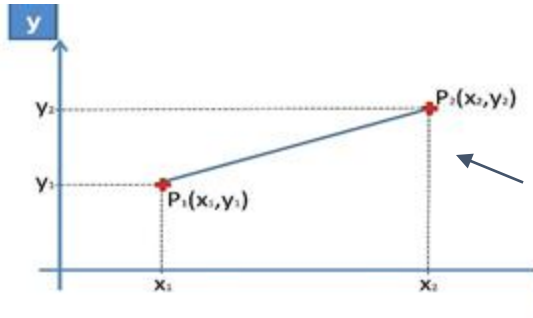
- Best for continuous numeric data with few dimensions.
- Magnitude/size of differences matters- larger differences have greater impact.
- Sensitive to outliers and high D data.
- Need to normalize/scale the data.

$$\text{Euclidean distance} = \sqrt{\sum_{i=1}^{n}\left(x_i - y_i\right)^2}$$



$$d(x, y) = \sqrt{\sum_{i=1}^{n}(y_i - x_i)^2}$$





Consider differences along each dimension

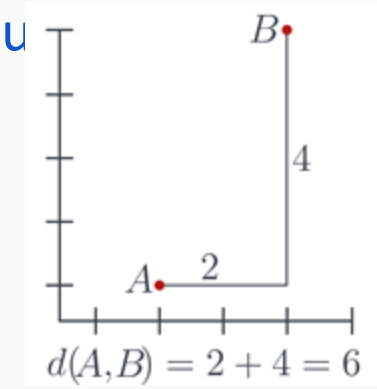
Euclidean Distance between P_1 and $P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

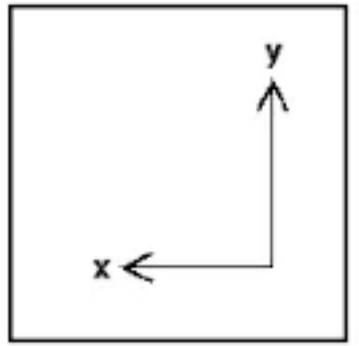
Manhattan Distance (L1 or city block or taxicab distance)

Measures the distance between two points on a grid, considering only horizontal and vertical movements (no diagonals- mimicking a taxicab navigating city streets).

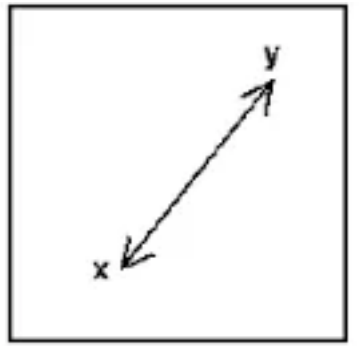
- Preferred in high-dimensional spaces: reduces the influence of irrelevant dimensions by focusing on absolute differences
- Formula: Takes the sum of the absolu between each feature:

$$d = \sum_{i=1}^{n} |x_i - y_i|$$





Manhattan



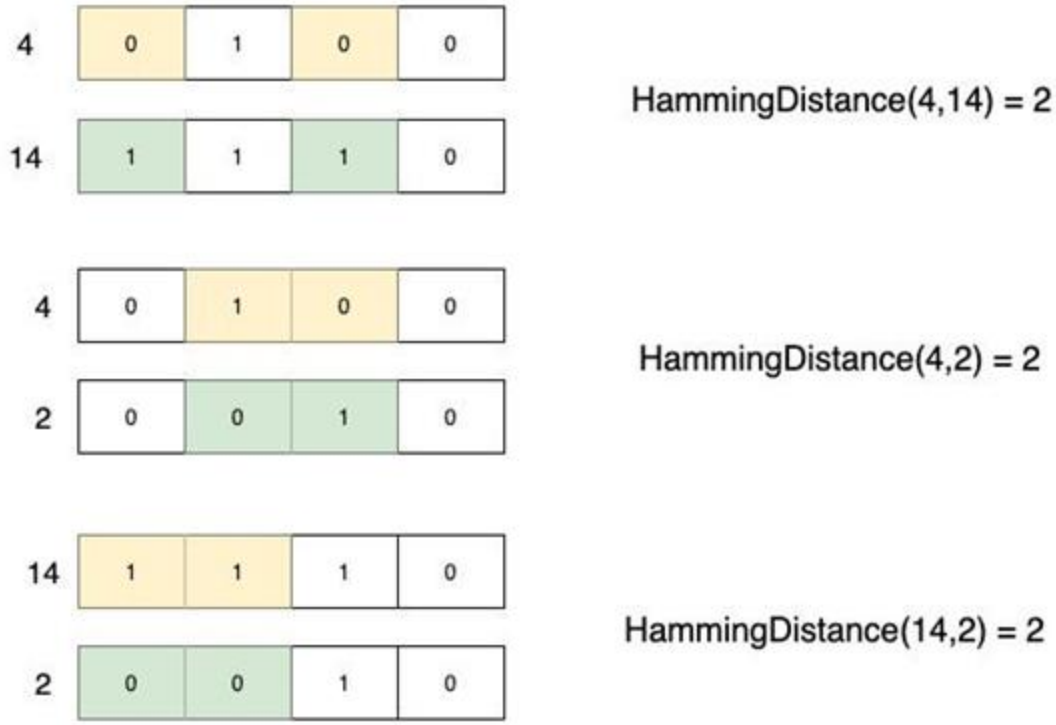
Euclidean



Hamming Distance

Categorical/Binary Data: measures the difference between two strings of equal length.

- Given two binary strings, count the number of different bits.
  - Example: Commonly applied in text classification and DNA sequence analysis.



Cosine Similarity

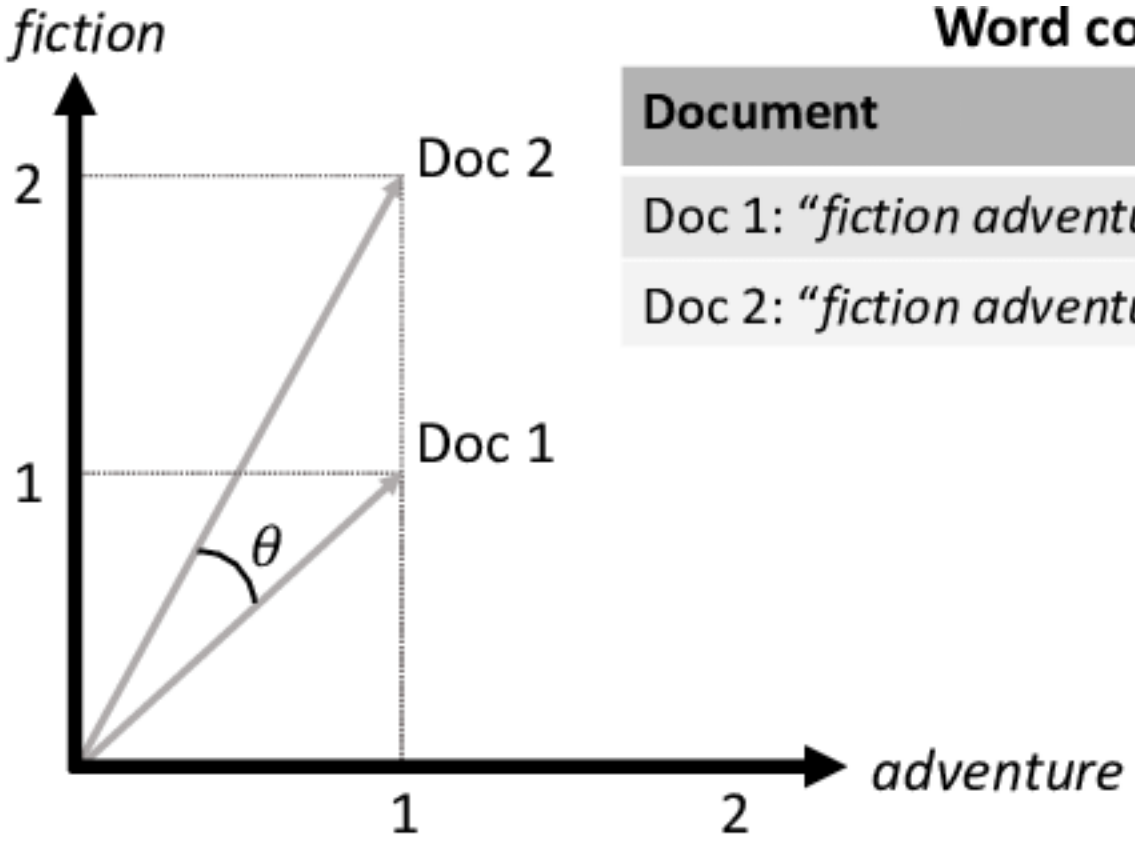
When to Use:

- **Ideal for Text Data:** (e.g., TF-IDF vectors) measures the angle between vectors, ignoring their magnitude.
- **Works well for Sparse Data (**most features are zero)
- **Direction Matters:** the orientation of vectors is more important than their magnitude (e.g., document similarity).

- **Example Use Cases:**
  a. Document similarity (e.g., comparing articles or reviews).
  b. Recommendation systems (e.g., user-item interactions).
  c. Clustering text data (e.g., grouping similar tweets)**.**

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Cosine Similarity



Word count per document

Document	*fiction*	*adventure*
Doc 1: "*fiction adventure*"	1	1
Doc 2: "*fiction adventure fiction*"	2	1

$$\cos\theta = \frac{\overrightarrow{Doc\ 1} \cdot \overrightarrow{Doc\ 2}}{\left\|\overrightarrow{Doc\ 1}\right\| \left\|\overrightarrow{Doc\ 2}\right\|}$$

$$= \frac{(1 * 2) + (1 * 1)}{\sqrt{1^2 + 1^2}\sqrt{2^2 + 1^2}}$$

$$= 0.95$$

- **Range:** Values range from -1 (completely dissimilar or pointing in opposite directions) to 1 (identical or similar in direction).

Summary: The choice of distance metric

Depends on the problem, data type, and dimensionality.

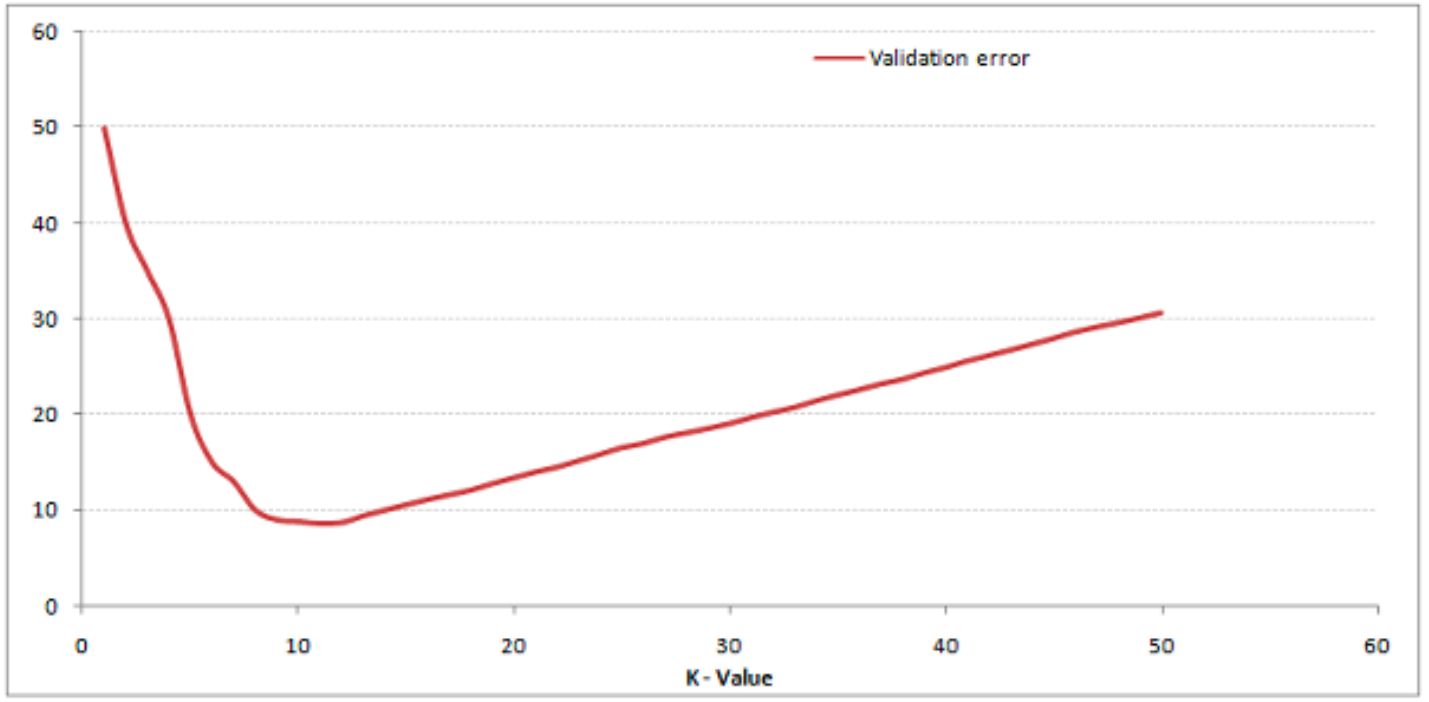
- **Manhattan:** High-dimensional numerical data, robust to outliers.
- **Euclidean:** Low-dimensional numerical data, magnitude matters.
- **Hamming:** Categorical/binary data.
-

```
# Importing necessary libraries
from scipy.spatial import distance # To calculate distances
```

How to Choose K in KNN

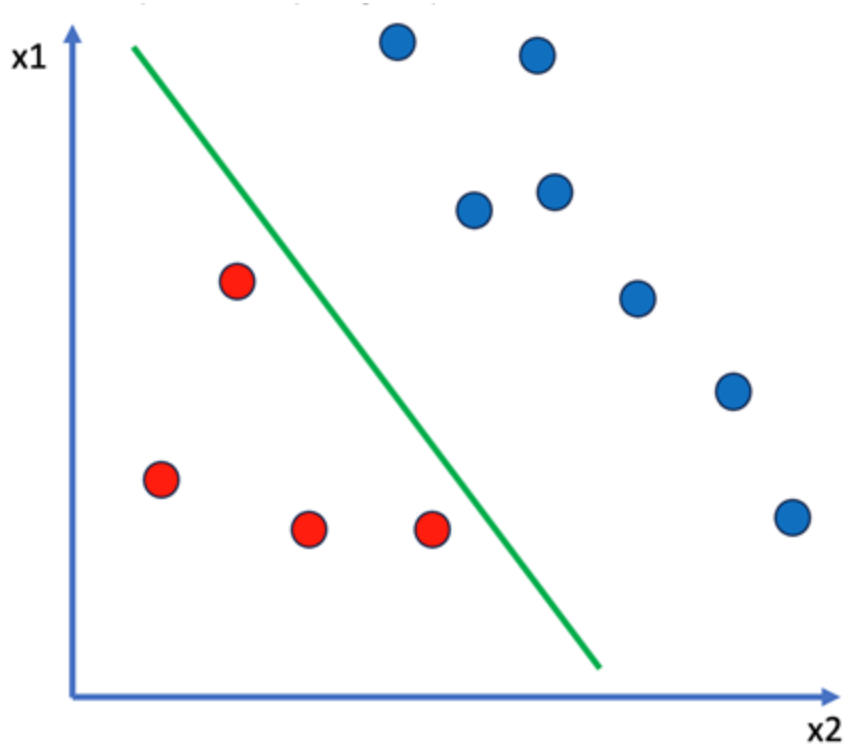
Try multiple Ks and graph the error rate, **pick the lowest**!

- This approach is known as the **"elbow method"** and can be an effective way to identify an appropriate K value for your specific problem.



Decision Boundaries: Training Models With the Goal of Generalization:

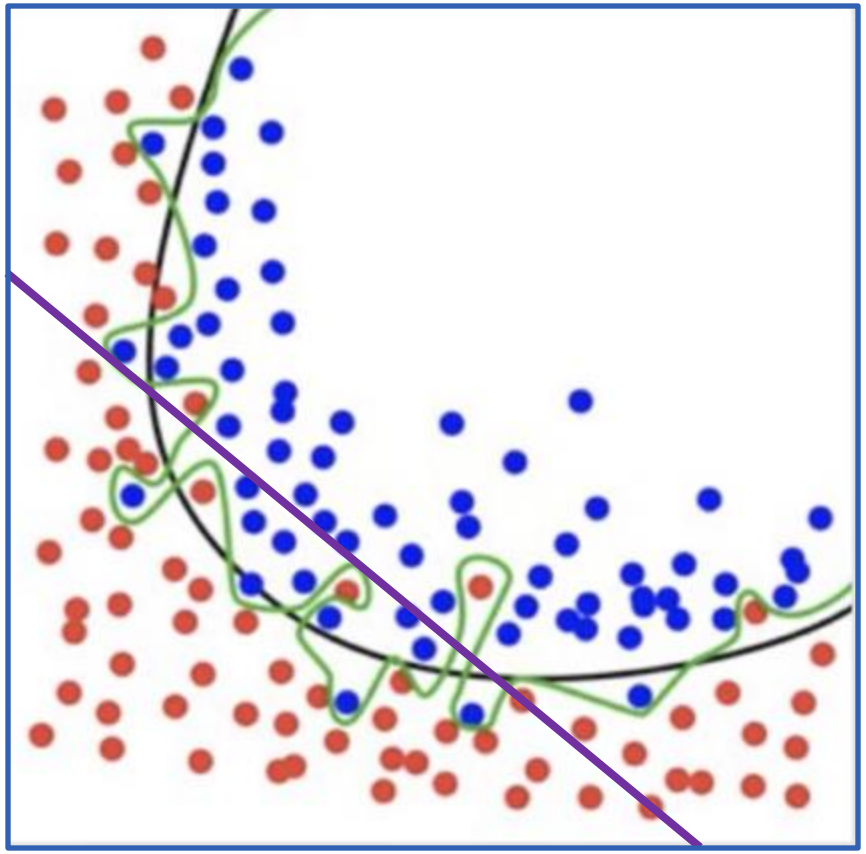
Is the line that separates positive and negative regions in the feature space



1 squiggly line
overfit

2 straight line

3 curve
good



Decision Boundary of a Classifier

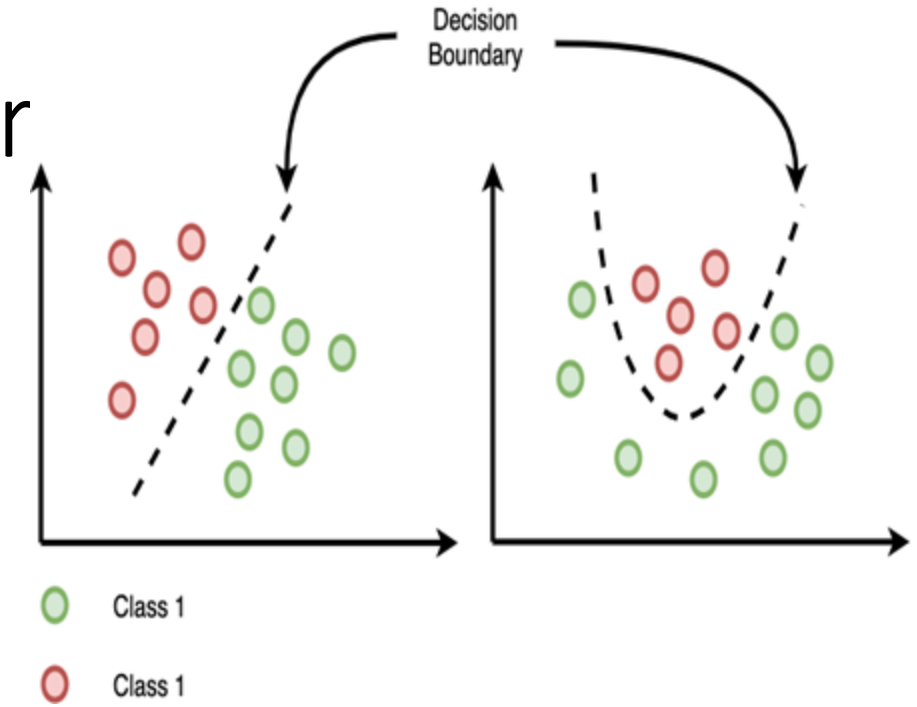
Purpose: It determines how new data points are classified based on their features.

Types:

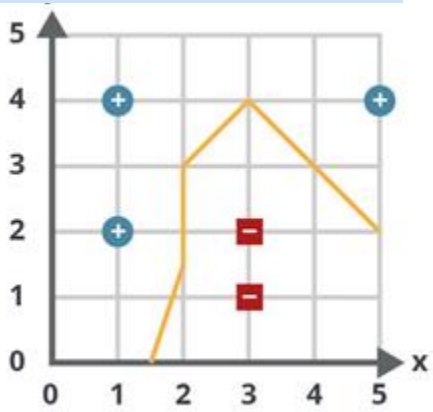
- **Linear Boundary:** Straight line that separates classes in a linearly separable problem.
- **Non-linear Boundary:** Complex shapes that separate classes in non-linearly separable problems.

Why is it useful? It helps us

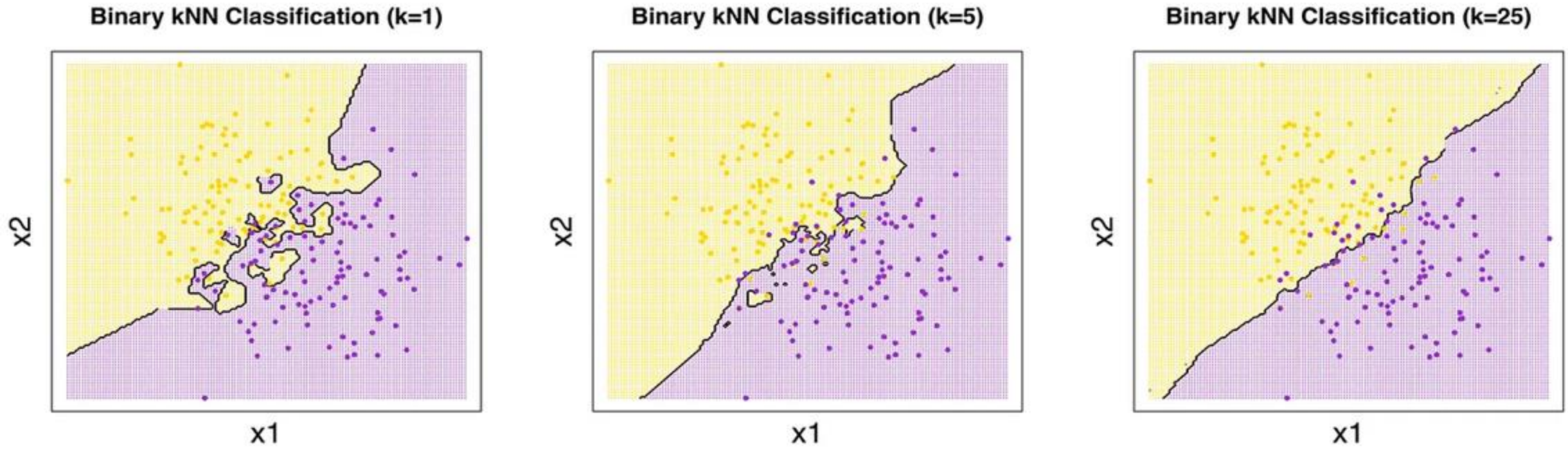
- visualize how examples will be classified for the entire feature space
- visualize the complexity of the learned model



For KNNs, we can answer the question: for any point in space, to which class does that new point belong?



Boundary Condition of KNN



K=1: Ignores broader patterns, focusing only on the closest point.
Decision Boundary: Irregular and jagged—highly sensitive to individual data points.

- **Behavior:** Overfits—memorizes training data and lacks generalization.

K=N (All Data Points as Neighbors)
Decision Boundary: Smooth and overly simplistic.

- **Behavior:** Underfits—ignores local structure and treats the entire dataset as a single group.

Predicts the mode (classification) or average (regression) for all new points.

Python Code Sample for KNN

- First Import necessary modules

```
from sklearn.neighbors import KNeighborsClassifier
```

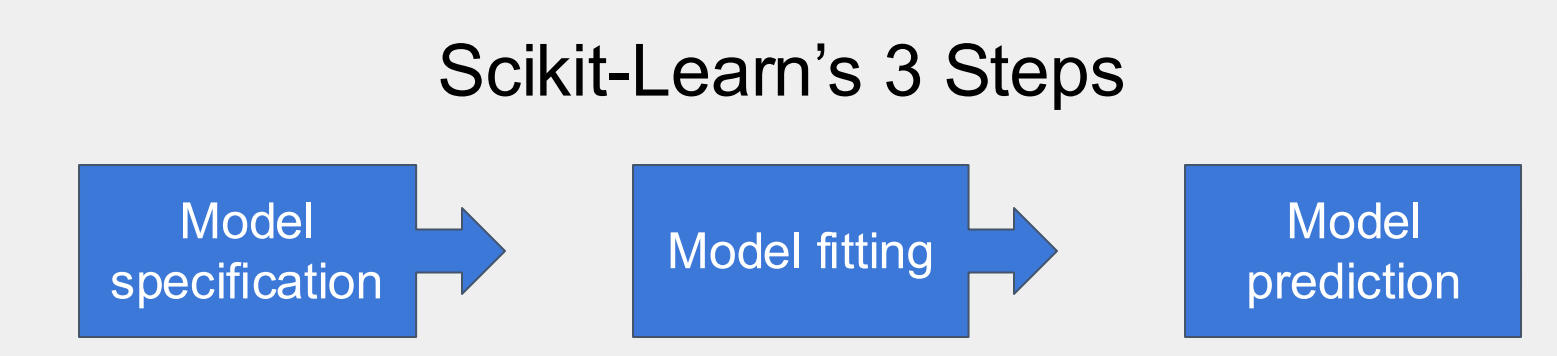
- Then you have load data, create feature and target/label
- Next you split into training and test set
- Then apply KNN:

```
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)
```

- Once training is done, predict on the test dataset which model has not seen before. Check .predict() function.

K-Nearest Neighbors in Python Code




```
# Initialize the model
model = KNeighborsClassifier(n_neighbors = 3)

# Train the model using training sets
model.fit(X_train, y_train)

# Make predictions on the test set
prediction = model.predict(X_test)

# Compute and print model's accuracy score
score = accuracy_score(y_test, prediction)
print('Accuracy score of the model: ' + str(score))
```

Pros and Cons of KNN

Pros	Cons
1. **Simple and Intuitive**: Easy to understand and implement. 2. **No Training Phase (Lazy Learning)**: Doesn't require explicit training—just stores the data. 3. **Adapts to Data**: Can model complex, non-linear relationships. 4. **Versatile**: Works for both **classification** and **regression**. 5. **No Assumptions**: Doesn't assume any underlying data distribution.	1. **Lazy Learning Drawbacks**: ○ **Computationally Expensive at Prediction Time**: Requires calculating distances to all training points for every prediction. ○ **Memory Intensive**: Stores the entire dataset, which can be inefficient for large datasets. 2. **Sensitive to Irrelevant Features**: Performance degrades with irrelevant or noisy features. 3. **Curse of Dimensionality**: Struggles in high-dimensional spaces. 4. **Choice of K and Distance Metric**: Requires careful tuning of K and the distance metric. 5. **Slow for Large Datasets**: Prediction time increases with the size of the dataset.

Summary:

- **Supervised Learning:** The model learns from labeled training data (e.g., "Paid back loan" or "Did not pay back loan").
- **Classification:** We categorize data into groups, deciding if someone falls into "Pay back" or "Not".
- **Binary Classification:** Only two categories exist (e.g., Pay back or not).
- KNN is an **instance based model** meaning it requires all the training data be stored to work.
    - This type of model makes decisions based on similar examples from the training data.